ssGSEA3.0: from permutation testing to feature-level bootstrap resampling

Contents

Overview . . . 1
Why bootstrap instead of permutation? . . . 2
Helpers used below . . . 2
Example 1: ssGSEA on LUAD NMF factors, HALLMARK . . . 4
Example 2: the same LUAD factors, REACTOME . . . 5
Example 3: PTM-SEA on a PI3K-inhibitor perturbation dataset . . . 8
Session info . . . 10

Overview

`ssGSEA3.0` runs single-sample Gene Set Enrichment Analysis (ssGSEA) on gene-centric expression data, and PTM-Signature Enrichment Analysis (PTM-SEA) on site-centric phosphoproteomics data scored against the directional PTM signatures database (PTMsigDB). Significance is assessed by feature-level (gene- or site-level) bootstrap resampling, replacing the gene-label permutation testing used by `ssGSEA2.0`.

This vignette reproduces the biological comparisons from the ssGSEA3.0 manuscript (Clark and Mani), using the same three implementations throughout:

- ssGSEA2.0-orig – `ssGSEA2.0` as published (Krug et al. 2019);
- ssGSEA2.0-fixed – `ssGSEA2.0` with its correlation-vector reordering fixed, available on the `ssGSEA2.0-fixed` branch of this repository and archived at doi:10.5281/zenodo.23734330;
- ssGSEA3.0 – this package, archived at doi:10.5281/zenodo.23734262.

It covers:

1. why the bootstrap replaced permutation,
2. ssGSEA on lung adenocarcinoma (LUAD) NMF factors against the MSigDB HALLMARK collection (manuscript Supplementary Figure 1),
3. the same factors against the larger, more diverse REACTOME collection (manuscript Figure 2), and
4. directional PTM-SEA on a PI3Kα-inhibitor phosphoproteomics dataset (manuscript Figure 3).

```
library(ssGSEA3)
```

About the bundled results

The results loaded below, bundled with this package in `inst/extdata/`, are the exact outputs reported in the manuscript and its Supplementary Tables 1-3. If you run the `ssGSEA3()` calls shown below with the current package, your numbers will differ slightly from these, and the differences are due to bootstrap randomness alone. The scoring and significance algorithm is the same.

The manuscript's ssGSEA3.0 runs were made before commit `7303e16`. Before that commit, `par = TRUE` gave each parallel worker its own random-number stream, and which gene sets a worker scored depended on timing, so a seed did not fix which random draws each gene set's bootstrap used. The current package gives every gene set its own stream, so `seed = 42` now reproduces the same results on every run. Those results are a different random draw from the manuscript's, so counts near the significance threshold shift slightly (for example, 805 rather than 808 significant REACTOME gene sets at `weight = 1`), and the conclusions are unchanged. `ssGSEA2.0` has no seed parameter, so its results also vary slightly from run to run for the same reason.

Why bootstrap instead of permutation?

Weighted scoring under permutation is miscalibrated

ssGSEA computes a running-sum enrichment score in which each feature's contribution is scaled by a `weight` parameter: `weight = 0` is a rank-only score, and `weight > 0` favors features with larger expression magnitude. `ssGSEA2.0-orig` assesses significance by permuting gene labels, and in doing so reorders each feature's weight (the "correlation vector") together with its rank. For `weight > 0` this makes highly expressed features drive the permuted scores, giving a null distribution that is too narrow and inflating false positives (GitHub issue #45 against `ssGSEA2.0`).

Scoring 400 randomly constructed (null) gene sets against the LUAD data shows this directly. At `weight = 0.75`, `ssGSEA2.0-orig` calls 11.6% of null gene sets significant at $p < 0.05$, against an expected 5%. Holding the weight vector fixed in its observed order and permuting only gene-set membership (`ssGSEA2.0-fixed`) brings this back to 5.2%. `ssGSEA3.0` gives 6.25%, and its null p-values are the only ones at `weight = 0.75` not significantly different from uniform (Kolmogorov-Smirnov $p = 0.44$).

The fix costs detection power; the bootstrap recovers it

`ssGSEA2.0-fixed` restores calibration, but finds noticeably fewer significant pathways at nonzero weight – including biologically relevant ones (see the REACTOME example below). In a synthetic simulation with a known embedded ±0.5 shift (10,000 gene sets of 20 members, `weight = 1`), `ssGSEA3.0` detected significantly more true positives than `ssGSEA2.0-fixed` (4,629 vs. 4,211; odds ratio 3.11, McNemar $p < 0.0001$). `ssGSEA3.0` replaces gene-label permutation with bootstrap resampling of the measured features themselves, sampled with replacement. Each bootstrap draw is re-ranked by its resampled values and the gene set's members are relocated within that ranking, while each feature's weight is always taken from the original data in its original order. The bootstrap scores are centered to their own mean, their standard deviation is used as a bootstrap standard error, and the p-value comes from the resulting z-score. NES is the observed score divided by the mean of the same-signed centered bootstrap scores.

Directional (PTM-SEA) scoring

PTM-SEA's directional gene sets (`;u`/`;d`-labeled "up" and "down" members) are scored by combining separately computed up and down sub-scores; with `ssGSEA3.0` both are rescored from the same bootstrap draw. In a synthetic simulation with a known embedded ±0.5 shift (10,000 gene sets of 20 members, `weight = 0`), both permutation and bootstrap were well-calibrated under the null, and the bootstrap found significantly more true positives for both non-directional (428 bootstrap-only vs. 229 permutation-only; odds ratio 1.87, McNemar $p < 0.0001$) and directional gene sets (390 vs. 296; odds ratio 1.32, McNemar $p = 0.0004$). On real data the gain is much larger for directional scoring, as Example 3 shows.

Helpers used below

All three examples count a gene set as significant if it passes the FDR threshold in at least one sample (not by counting gene-set x sample cells, which would double-count gene sets significant in several samples), and draw side-by-side heatmaps with one shared row order and color scale so the panels can be compared directly:

```r
## gene sets significant (FDR < fdr.max) in >= 1 sample
is_sig <- function(res, fdr.max) apply(res$fdr < fdr.max, 1, any)

## pooled NES correlation across all gene sets and samples
nes_cor <- function(a, b, method = "pearson") {
  cor(as.vector(a$scores), as.vector(b$scores), method = method)
}

## one plot_ssgsea_heatmap() panel per implementation, sharing the first
## panel's row clustering and one color scale (1st/99th percentile clamp of
## the plotted NES across all panels). `labels` optionally shortens long
## gene set names for display.
compare_heatmaps <- function(results, terms, fdr.max, labels = NULL) {
  if (!is.null(labels)) {
    new <- setNames(labels(terms), terms)
    results <- lapply(results, function(r) {
      r <- r[c("scores", "fdr", "sample.names", "output.score.type")]
      r$scores <- r$scores[terms, , drop = FALSE]
      r$fdr <- r$fdr[terms, , drop = FALSE]
      rownames(r$scores) <- rownames(r$fdr) <- r$gs.names <- unname(new[terms])
      r
    })
    terms <- unname(new[terms])
  }
  nes <- unlist(lapply(results, function(r) r$scores[terms, ]))
  q <- quantile(nes, c(0.01, 0.99))
  nes.range <- c(floor(q[1]), ceiling(q[2]))
  first <- results[[1]]$scores
  shared.clustering <- function(m) hclust(dist(first[rownames(m), , drop = FALSE]))
  n <- length(results)
  panels <- lapply(seq_len(n), function(i) {
    ht <- plot_ssgsea_heatmap(results[[i]], terms = terms, fdr.max = fdr.max,
                              cluster_rows = shared.clustering, nes.range = nes.range,
                              title = names(results)[i], title_fontsize = 10,
                              show_category = i == 1, show_row_dend = i == 1,
                              show_row_names = i == n, show_heatmap_legend = i == n)
    ## concatenated heatmaps need unique names; only the last panel's ("NES")
    ## is ever shown, as its legend title
    if (i < n) ht@name <- paste0("NES_", i)
    ## angled, so titles wider than their narrow panels don't collide
    ht@column_title_param$rot <- 45
    ht@column_title_param$just <- c(0, 0.5)
    ht
  })
  ComplexHeatmap::draw(Reduce(`+`, panels), ht_gap = grid::unit(6, "mm"))
}

## REACTOME names run past 70 characters: drop the prefix and underscores,
## and cut long names at a word boundary
short_reactome <- function(x, max.chars = 42) {
  x <- gsub("_", " ", sub("^REACTOME_", "", x))
  cut <- sub("\\s+\\S*$", "", substr(x, 1, max.chars))
  ifelse(nchar(x) > max.chars, paste0(cut, "..."), x)
```

```
}

method.names <- c(orig = "ssGSEA2.0-orig", fixed = "ssGSEA2.0-fixed", v3 = "ssGSEA3.0")
```

Example 1: ssGSEA on LUAD NMF factors, HALLMARK

The bundled LUAD dataset holds the feature weights from non-negative matrix factorization (NMF) of a lung adenocarcinoma cohort (Satpathy et al. 2025), derived with the PANOPLY pipeline. Its four factors are named after their published biological identities: Unstable-proliferative, Quiescent-EGFR, Immune-active-KRAS and Stable-early-stage. The original publication scored these against MSigDB HALLMARK v6.2 at `weight = 1` and FDR < 0.01:

```
res <- ssGSEA3(
  input.ds = system.file("example", "LUAD_matrix_W_combined_signed_gc.gct", package =
   ↪  "ssGSEA3"),
  output.prefix = "luad",
  gene.set.databases = system.file("example", "h.all.v6.2.symbols.gmt", package =
   ↪  "ssGSEA3"),
  sample.norm.type = "rank", weight = 1, correl.type = "z.score",
  statistic = "area.under.RES", min.overlap = 10, nperm = 1000, seed = 42
)
```

This takes about 0.8 minutes on 10 cores (`ssGSEA2.0-orig`: about 0.5 minutes). Here we load the manuscript's results for all three implementations, at both `weight = 1` and `weight = 0`:

```
luad <- readRDS(system.file("extdata", "luad_hallmark_manuscript_results.rds",
                            package = "ssGSEA3"))
luad$weight1$v3
#> <ssgsea_result> 50 gene sets x 4 samples
#>   score type: NES
fdr.hallmark <- 0.01

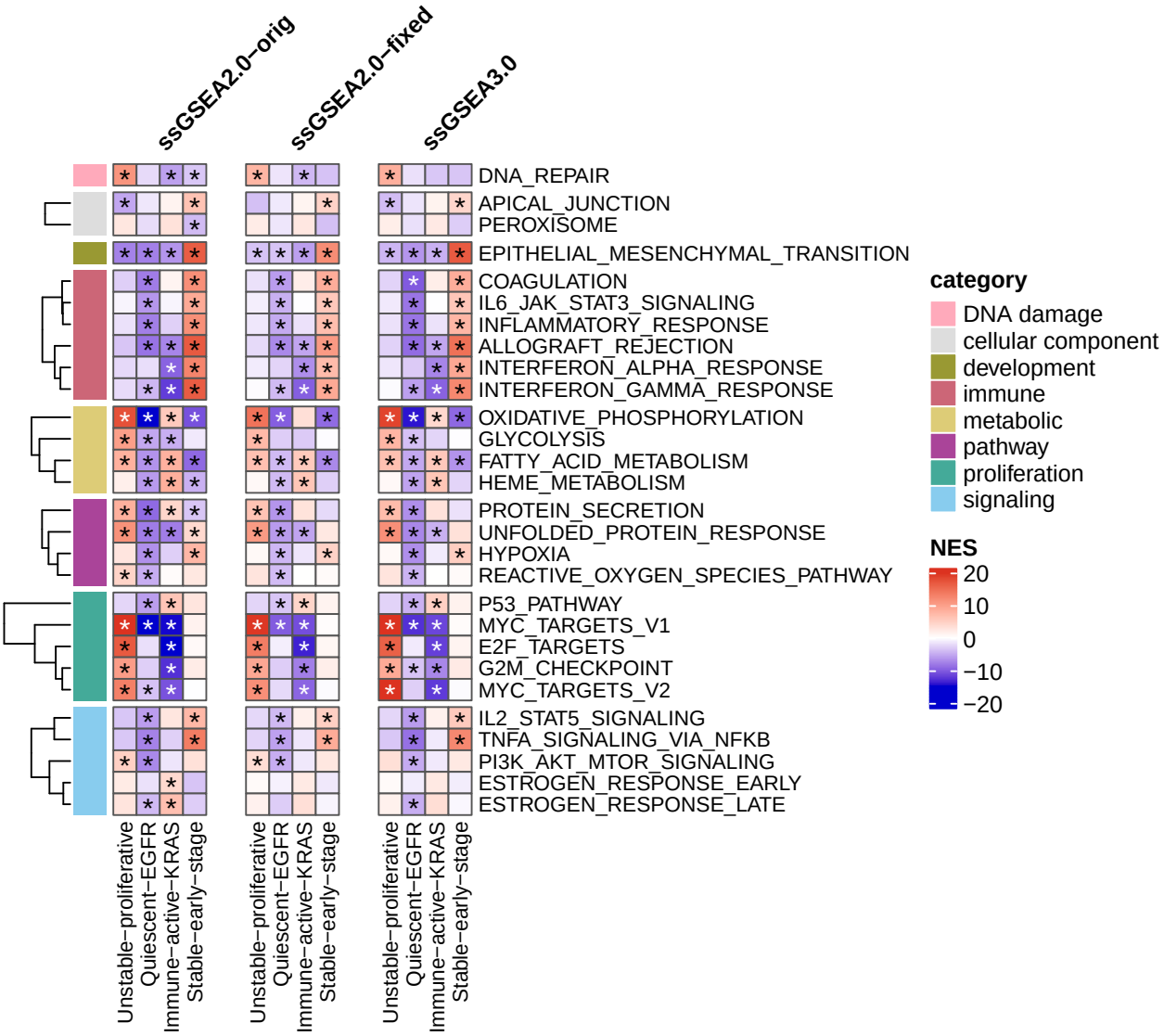
sig.hallmark <- sapply(c("weight0", "weight1"), function(w) {
  sapply(luad[[w]], function(r) sum(is_sig(r, fdr.hallmark)))
})
dimnames(sig.hallmark) <- list(method.names, c("weight = 0", "weight = 1"))
sig.hallmark
#>                 weight = 0 weight = 1
#> ssGSEA2.0-orig          38         44
#> ssGSEA2.0-fixed         38         36
#> ssGSEA3.0               38         38
```

At `weight = 0`, where the correlation-vector reordering cannot occur, all three implementations find 38 significant gene sets. At `weight = 1`, fixing the reordering drops `ssGSEA2.0-orig`'s 44 to 36, while `ssGSEA3.0` finds 38, keeping 2 that `ssGSEA2.0-fixed` lost. The results nonetheless agree closely: 36 of the 36 gene sets significant in `ssGSEA2.0-fixed` are also significant in `ssGSEA2.0-orig`, and pooled NES correlations at `weight = 1` are high:

```
round(c(
  "orig vs. fixed" = nes_cor(luad$weight1$orig, luad$weight1$fixed),
  "orig vs. ssGSEA3.0" = nes_cor(luad$weight1$orig, luad$weight1$v3)
), 2)
#>     orig vs. fixed orig vs. ssGSEA3.0
#>               0.98               0.95
```

The 28 curated HALLMARK gene sets shown in the original publication look consistent across all three implementations (∗ marks FDR < 0.01):

```
compare_heatmaps(setNames(luad$weight1, method.names), luad$heatmap.terms$hallmark,
                 fdr.max = fdr.hallmark)
```



So the strongest signals found by `ssGSEA2.0-orig` are not artifacts of the reordering, and previously published `weight = 1` results remain largely reliable.

Example 2: the same LUAD factors, REACTOME

HALLMARK's 50 gene sets are curated to be similar in size and narrow in scope. REACTOME (1,167 gene sets after the `min.overlap = 10` filter, ranging from ten to several hundred genes) is a harder test of the same data, scored at FDR < 0.05:

```
res.reactome <- ssGSEA3(
  input.ds = system.file("example", "LUAD_matrix_W_combined_signed_gc.gct", package =
   ↪  "ssGSEA3"),
  output.prefix = "luad_reactome",
```

```
  gene.set.databases = system.file("example", "reactome_subset.gmt", package =
   ↪  "ssGSEA3"),
  sample.norm.type = "rank", weight = 1, correl.type = "z.score",
  statistic = "area.under.RES", min.overlap = 10, nperm = 1000, seed = 42
)
```

This takes about 18 minutes on 10 cores (`ssGSEA2.0-orig`: about 9 minutes).

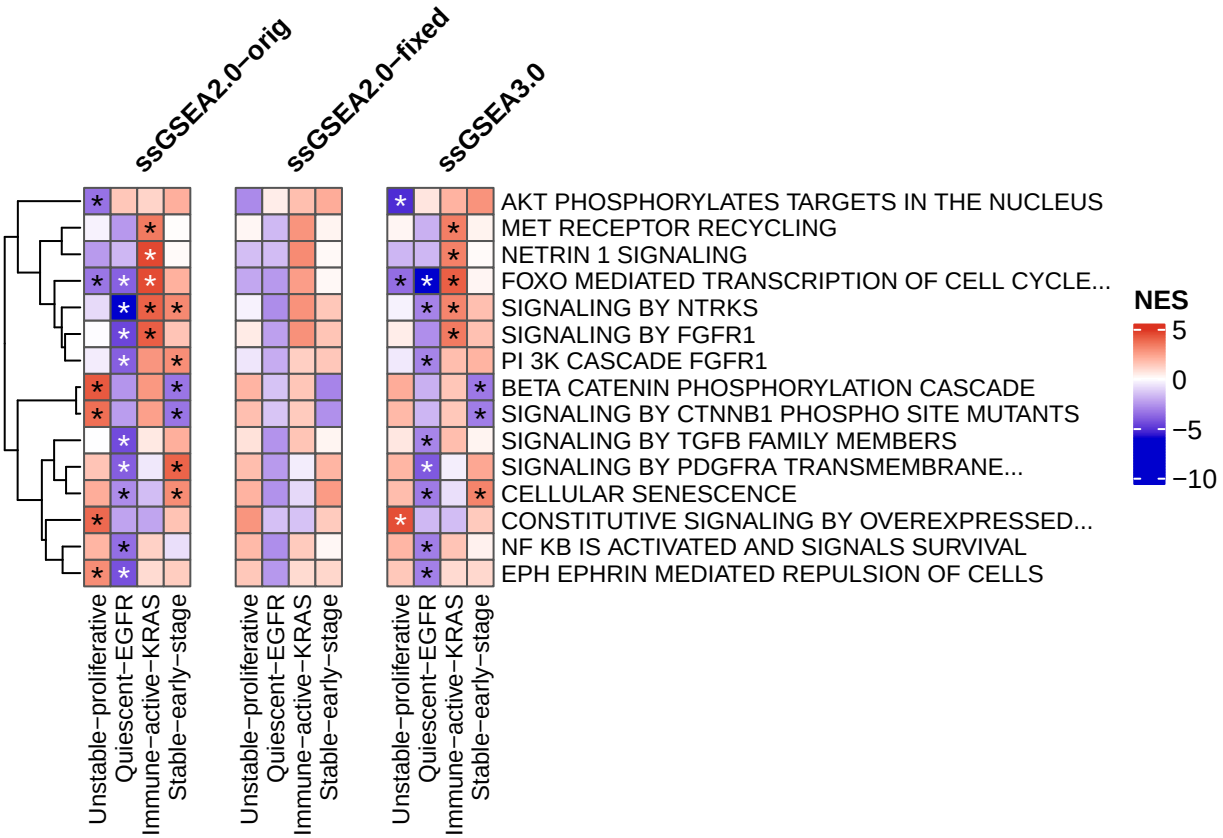
```
reactome <- readRDS(system.file("extdata", "luad_reactome_manuscript_results.rds",
                                package = "ssGSEA3"))
fdr.reactome <- 0.05

sig.reactome <- sapply(c("weight0", "weight1"), function(w) {
  sapply(reactome[[w]], function(r) sum(is_sig(r, fdr.reactome)))
})
dimnames(sig.reactome) <- list(method.names, c("weight = 0", "weight = 1"))
sig.reactome
#>                 weight = 0 weight = 1
#> ssGSEA2.0-orig         819       1019
#> ssGSEA2.0-fixed        819        734
#> ssGSEA3.0              878        808

round(c(
  "orig vs. fixed" = nes_cor(reactome$weight1$orig, reactome$weight1$fixed),
  "orig vs. ssGSEA3.0" = nes_cor(reactome$weight1$orig, reactome$weight1$v3)
), 2)
#>     orig vs. fixed orig vs. ssGSEA3.0
#>               0.97               0.88
```

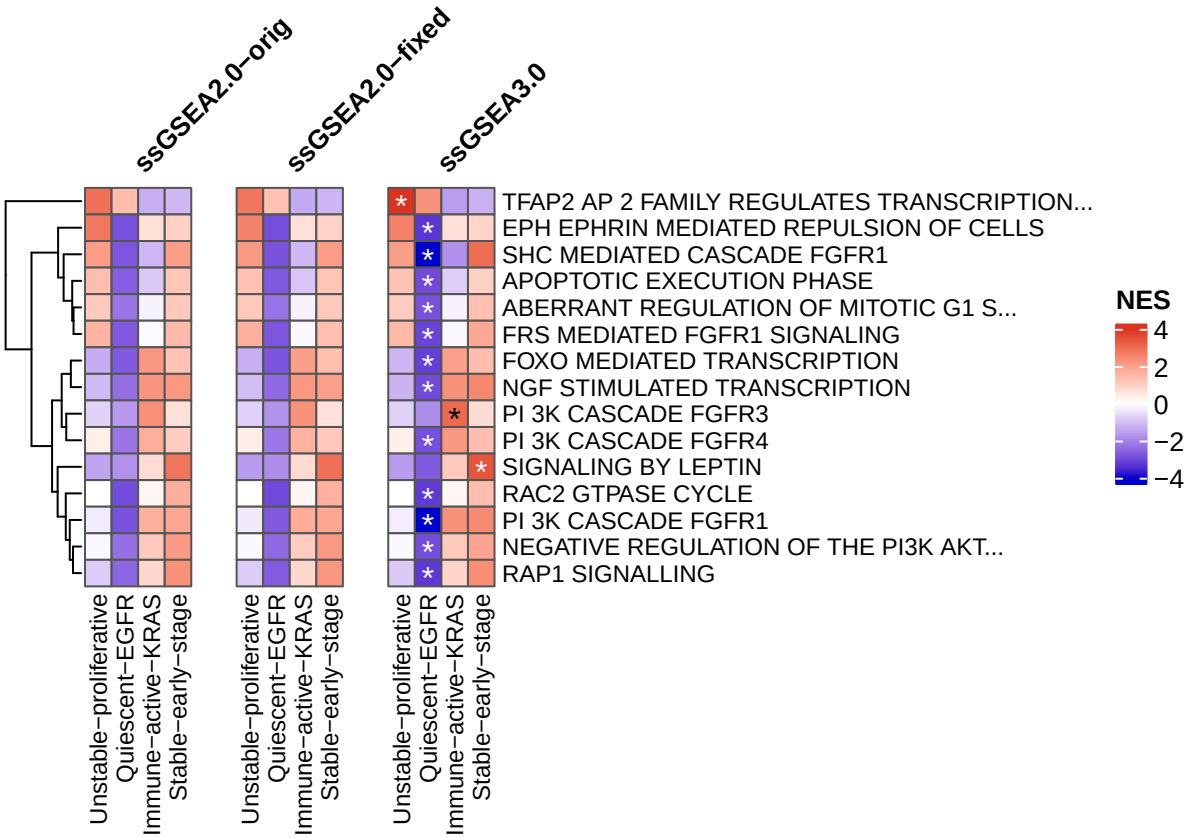
The pattern seen on HALLMARK is clearer here. Despite NES correlation of 0.97, `ssGSEA2.0-fixed` finds 28% fewer significant gene sets than `ssGSEA2.0-orig` at `weight = 1`. Almost all of its calls are shared with `ssGSEA2.0-orig` (733 of 734), but it misses 286, and some of those are relevant to LUAD, such as FGFR1 signaling and cellular senescence. `ssGSEA3.0` finds 808, a 10% increase over `ssGSEA2.0-fixed`. Fifteen representative gene sets significant in both `ssGSEA2.0-orig` and `ssGSEA3.0` but missed by `ssGSEA2.0-fixed` at `weight = 1` (manuscript Figure 2D; $*$ marks FDR < 0.05):

```
compare_heatmaps(setNames(reactome$weight1, method.names),
                 reactome$heatmap.terms$reactome.fixed.missed, fdr.max = fdr.reactome,
                 labels = short_reactome)
```



At `weight = 0`, both `ssGSEA2.0` implementations find 819 significant gene sets and `ssGSEA3.0` finds 878 (7% more), including PI3K signaling and aberrant cell-cycle regulation. Fifteen representative gene sets significant only in `ssGSEA3.0` at `weight = 0` (manuscript Figure 2E):

```
compare_heatmaps(setNames(reactome$weight0, method.names),
                 reactome$heatmap.terms$reactome.v3.only.w0, fdr.max = fdr.reactome,
                 labels = short_reactome)
```



So `ssGSEA3.0` fixes the calibration problem in `ssGSEA2.0-orig` without giving up as much detection power as `ssGSEA2.0-fixed`, and the `weight = 0` result suggests that feature-level bootstrapping itself has more power than gene-label permutation.

Example 3: PTM-SEA on a PI3K-inhibitor perturbation dataset

The second dataset (installed with the package in its `example/` folder) is site-centric phosphoproteomics from the original PTM-SEA publication (Krug et al. 2019 – see the package `README` for the full citation): T47D breast cancer cells treated with the PI3Kα inhibitor BYL719 vs. DMSO for 6h and 24h. The input values are log-signed p-values from a moderated two-sample t-test at each timepoint. Because these are not measured values, they are scored at `weight = 0` against PTMsigDB v2.0.0's human flanking-sequence signatures:

```
res.ptmsea <- ssGSEA3(
  input.ds = system.file("example", "PI3K_pert_logP_n2x23936.gct", package = "ssGSEA3"),
  output.prefix = "pi3k",
  gene.set.databases = system.file("example", "ptm.sig.db.all.flanking.human.v2.0.0.gmt",
   ↪  package = "ssGSEA3"),
  sample.norm.type = "rank", weight = 0, correl.type = "rank",
  statistic = "area.under.RES", min.overlap = 5, nperm = 1000, seed = 42
)
```

This takes about 9 minutes on 10 cores (`ssGSEA2.0-orig`: about 4 minutes). At `weight = 0` the two `ssGSEA2.0` implementations are equivalent, so as in the manuscript only `ssGSEA2.0-fixed` is compared with `ssGSEA3.0`:

```
pi3k <- readRDS(system.file("extdata", "pi3k_ptmsigdb_manuscript_results.rds",
                            package = "ssGSEA3"))
```

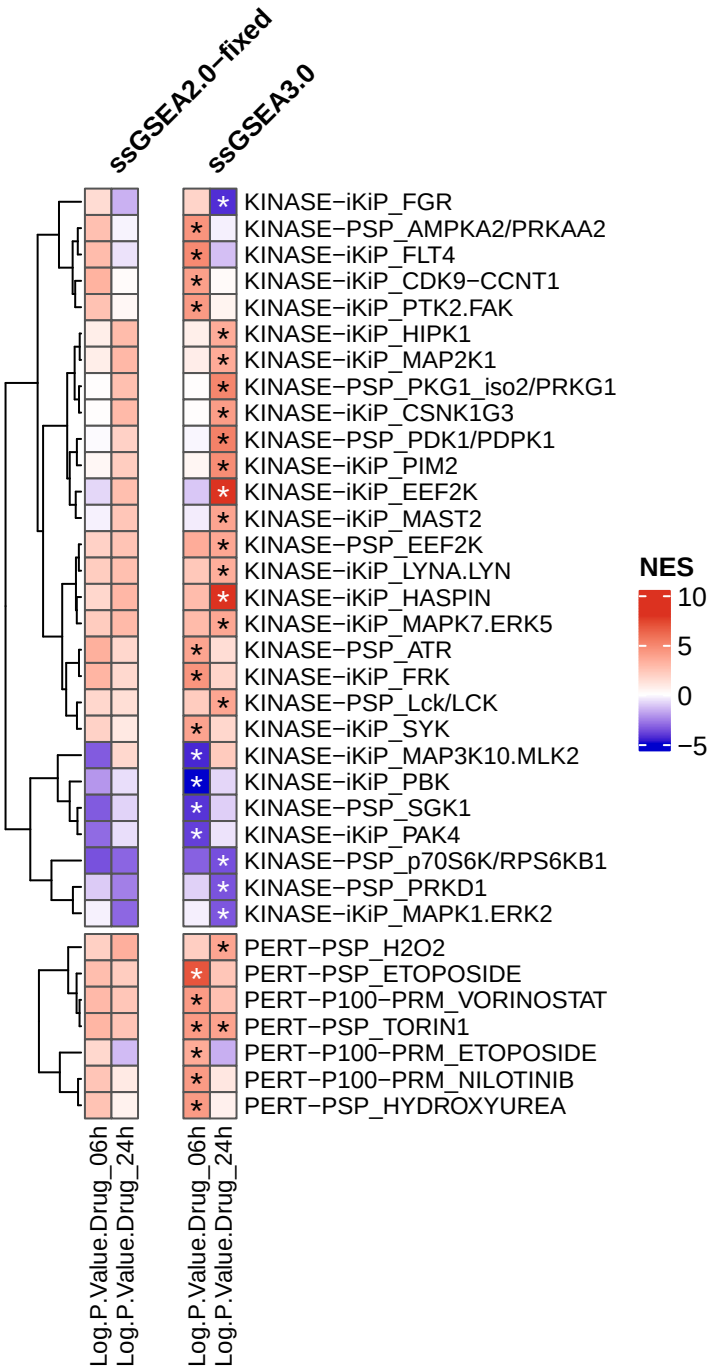
```
fdr.pi3k <- 0.05
p0 <- lapply(pi3k$weight0, is_sig, fdr.max = fdr.pi3k)

round(c(
  "NES Pearson r" = nes_cor(pi3k$weight0$fixed, pi3k$weight0$v3),
  "NES Spearman rho" = nes_cor(pi3k$weight0$fixed, pi3k$weight0$v3, method = "spearman")
), 2)
#>    NES Pearson r NES Spearman rho
#>             0.91             0.99

c("ssGSEA2.0-fixed" = sum(p0$fixed), "ssGSEA3.0" = sum(p0$v3),
  "both" = sum(p0$fixed & p0$v3), "ssGSEA3.0 only" = sum(p0$v3 & !p0$fixed))
#> ssGSEA2.0-fixed       ssGSEA3.0            both  ssGSEA3.0 only
#>              93             126              86              40
```

`ssGSEA3.0` keeps most of `ssGSEA2.0-fixed`'s significant signatures (86 of 93) while finding 35% more overall – far more than the 7% gain on REACTOME at `weight = 0`. Many of the added signatures sit on the PI3K signaling axis this experiment perturbs: upstream activators (PDK1, AMPKA2), downstream targets (p70S6K, SGK1), MAPK-pathway crosstalk, and the cell-cycle response. Thirty-five representative signatures significant in `ssGSEA3.0` but not `ssGSEA2.0-fixed` (manuscript Figure 3C; $*$ marks FDR < 0.05):

```
compare_heatmaps(setNames(pi3k$weight0, method.names[c("fixed", "v3")]),
                 pi3k$heatmap.terms$pi3k.v3.only, fdr.max = fdr.pi3k)
```



The added calls cluster around the perturbed pathway rather than scattering across unrelated biology, consistent with a genuine power advantage that is most pronounced for directional gene sets.

Session info

```
sessionInfo()
#> R version 4.4.2 (2024-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Sequoia 15.8.1
#>
```

```
#> Matrix products: default
#> BLAS:
 ↪  /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK:
 ↪  /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
 ↪  LAPACK version 3.12.0
#>
#> locale:
#> [1] C
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] ssGSEA3_1.0.0  testthat_3.3.2
#>
#> loaded via a namespace (and not attached):
#>   [1] remotes_2.5.0               rlang_1.1.7
#>   [3] magrittr_2.0.4              clue_0.3-66
#>   [5] GetoptLong_1.0.5            otel_0.2.0
#>   [7] verification_1.45           matrixStats_1.4.1
#>   [9] compiler_4.4.2              png_0.1-8
#>  [11] vctrs_0.6.5                 maps_3.4.3
#>  [13] profvis_0.4.0               pkgconfig_2.0.3
#>  [15] shape_1.4.6.1               crayon_1.5.3
#>  [17] fastmap_1.2.0               magick_2.9.1
#>  [19] XVector_0.46.0              ellipsis_0.3.2
#>  [21] promises_1.5.0              rmarkdown_2.30
#>  [23] sessioninfo_1.2.2           UCSC.utils_1.2.0
#>  [25] tinytex_0.58                purrr_1.2.1
#>  [27] xfun_0.55                   zlibbioc_1.52.0
#>  [29] cachem_1.1.0                GenomeInfoDb_1.42.1
#>  [31] jsonlite_2.0.0              later_1.4.5
#>  [33] DelayedArray_0.32.0         parallel_4.4.2
#>  [35] cluster_2.1.8               R6_2.6.1
#>  [37] RColorBrewer_1.1-3          boot_1.3-31
#>  [39] pkgload_1.4.0               brio_1.1.5
#>  [41] GenomicRanges_1.58.0        knitr_1.51
#>  [43] Rcpp_1.1.1                  SummarizedExperiment_1.36.0
#>  [45] iterators_1.0.14            fields_17.1
#>  [47] usethis_3.1.0               IRanges_2.40.1
#>  [49] flowCore_2.18.0             httpuv_1.6.16
#>  [51] Matrix_1.7-4                tidyselect_1.2.1
#>  [53] yaml_2.3.12                 rstudioapi_0.17.1
#>  [55] abind_1.4-8                 doParallel_1.0.17
#>  [57] codetools_0.2-20            dtw_1.23-1
#>  [59] miniUI_0.1.2                pkgbuild_1.4.5
#>  [61] lattice_0.22-6              tibble_3.3.1
#>  [63] Biobase_2.66.0              shiny_1.12.1
#>  [65] withr_3.0.2                 evaluate_1.0.5
```

```
#>  [67] desc_1.4.3                 proxy_0.4-29
#>  [69] urlchecker_1.0.1           circlize_0.4.16
#>  [71] pillar_1.11.1              MatrixGenerics_1.18.0
#>  [73] foreach_1.5.2              stats4_4.4.2
#>  [75] generics_0.1.4             rprojroot_2.1.1
#>  [77] S4Vectors_0.44.0           gtools_3.9.5
#>  [79] xtable_1.8-4               glue_1.8.0
#>  [81] tools_4.4.2                fs_1.6.6
#>  [83] dotCall64_1.2              grid_4.4.2
#>  [85] CircStats_0.2-7            RProtoBufLib_2.18.0
#>  [87] devtools_2.4.5             colorspace_2.1-2
#>  [89] GenomeInfoDbData_1.2.13    cmapR_1.18.0
#>  [91] cli_3.6.5                  spam_2.11-3
#>  [93] cytolib_2.18.2             S4Arrays_1.6.0
#>  [95] viridisLite_0.4.2          ComplexHeatmap_2.22.0
#>  [97] dplyr_1.1.4                digest_0.6.39
#>  [99] BiocGenerics_0.52.0        SparseArray_1.6.0
#> [101] rjson_0.2.23               htmlwidgets_1.6.4
#> [103] memoise_2.0.1              htmltools_0.5.9
#> [105] lifecycle_1.0.5            httr_1.4.7
#> [107] GlobalOptions_0.1.2        mime_0.13
#> [109] MASS_7.3-64
```